

RAPPORT DE PROJET

Développement d'une API REST en Python

BTS CIEL 2 – Année 2026/2027
BYLON Malick

RAPPORT DE PROJET

Développement d'une API REST en Python

Technologies utilisées :



1. Présentation du projet

Dans le cadre de ma formation en BTS CIEL, j'ai réalisé un projet consistant à développer une API REST en Python permettant de gérer une liste d'étudiants enregistrés dans une base de données MySQL nommée *ciel2027*.

Cette API, développée avec le framework Flask, permet de recevoir et de traiter des requêtes HTTP afin d'effectuer différentes opérations sur les données. La communication avec la base de données est réalisée grâce à la bibliothèque *mysql.connector*.

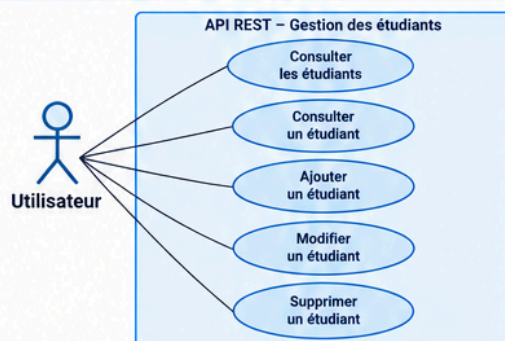
Les fonctionnalités principales sont :

- Consulter tous les étudiants
- Consulter un étudiant précis
- Ajouter un étudiant
- Modifier un étudiant
- Supprimer un étudiant

Paramètres de connexion :

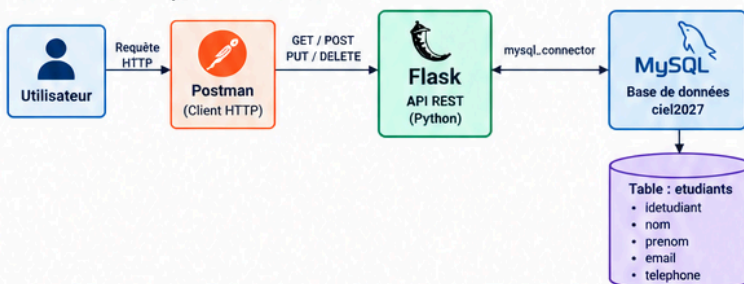
- Adresse : 127.0.0.1
- Utilisateur : root
- Base de données : ciel2027
- SGBD : MySQL

2. Diagramme de cas d'utilisation



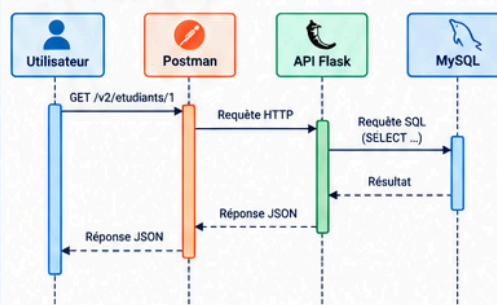
3. Architecture générale du projet

Le fonctionnement du projet repose sur plusieurs éléments : l'utilisateur envoie une requête HTTP depuis Postman. Flask reçoit la requête, exécute la fonction correspondant à la route demandée et communique avec la base de données MySQL grâce à *mysql.connector*. Le résultat est renvoyé au format JSON.



4. Diagramme de séquence – Consultation d'un étudiant

Ce diagramme montre les interactions entre l'utilisateur, Postman, l'API Flask et la base de données lors de la consultation d'un étudiant.



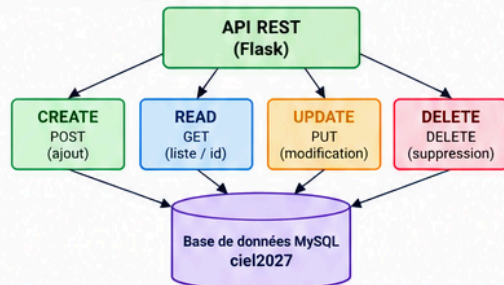
5. Description des routes de l'API

L'API utilise le protocole HTTP et renvoie les résultats au format JSON. Voici le détail des différentes routes disponibles :

Méthode	URL	Description	Corps de la requête (exemple)	Réponse	Codes HTTP
GET	/v2/etudiants/	Récupère la liste complète des étudiants.	-	[{ "idetudiant": 1, "nom": "Dupont", "prenom": "Jean", "email": "jean.dupont@univ.fr", "telephone": "0696000000" }]	200 OK
GET	/v2/etudiants/{id}	Récupère un étudiant par son identifiant.	-	Informations de l'étudiant	200 OK 404 Not Found
POST	/v2/etudiants/	Ajoute un nouvel étudiant.	{ "nom": "BYLON", "prenom": "Malick", "email": "malick@mail.fr", "telephone": "0696123456" }	Étudiant créé	201 Created
PUT	/v2/etudiants/{id}	Modifie les informations d'un étudiant.	{ "nom": "BYLON", "prenom": "Malick", "email": "malick@mail.fr", "telephone": "0696123456" }	Étudiant modifié	200 OK 404 Not Found
DELETE	/v2/etudiants/{id}	Supprime un étudiant.	-	Étudiant supprimé	200 OK 404 Not Found

6. Diagramme CRUD

L'API permet d'effectuer les quatre opérations CRUD (Create, Read, Update, Delete) sur la base de données des étudiants.



7. Diagramme de classes – Version 3

Une classe Database est prévue pour la version 3 afin de centraliser les opérations sur la base de données.

```
class Database:
    - host : str
    - user : str
    - password : str
    - database : str

    + connect() : void
    + getEtudiants() : list
    + getEtudiant(id : int) : dict
    + createEtudiant(data : dict) : bool
    + updateEtudiant(id : int, data : dict) : bool
    + deleteEtudiant(id : int) : bool
    + login(username : str, password : str) : bool
```

```
class MySQL:
    - host : str
    - user : str
    - password : str
    - database : str

    + tables : list
    + etudiants : list
    + user (prévu)
```

```
Table etudiants
- idetudiant
- nom
- prenom
- email
- telephone
```

8. Historique des versions

Version 1

- Implémentation des opérations CRUD.
- Routes avec le préfixe /v1/.

Version 2

- Gestion des erreurs avec try/except.
- Erreurs 404 et 500.
- Contrôle des données reçues.
- Vérification du rowcount.
- Routes avec le préfixe /v2/.

Version 3 (prévue)

- Création d'une table user.
- Authentification (POST /login).
- Création d'une classe Database.
- Programmation orientée objet.
- Séparation du code dans un fichier db.py.

9. Tests réalisés avec Postman

Test	URL	Résultat
GET tous les étudiants	/v2/etudiants/	200 OK
GET étudiant existant	/v2/etudiants/1	200 OK
GET étudiant inexistant	/v2/etudiants/999	404 Not Found
POST ajout étudiant	/v2/etudiants/	201 Created
PUT modification étudiant	/v2/etudiants/1	200 OK
DELETE étudiant	/v2/etudiants/1	200 OK

10. Compétences mises en œuvre

- Développement Python
- Framework Flask
- Base de données MySQL
- Requetes SQL (SELECT, INSERT, UPDATE, DELETE)
- Format d'échange JSON
- Utilisation de Postman
- Gestion des erreurs et codes HTTP
- Notions de programmation orientée objet (prévu)

11. Conclusion

Ce projet m'a permis de développer une API REST complète en Python avec Flask et MySQL. Les opérations CRUD ont été mises en place avec succès et les tests réalisés avec Postman ont permis de vérifier le bon fonctionnement de l'API ainsi que la gestion des erreurs. La version 3 permettra d'aller plus loin avec l'authentification utilisateur et une organisation du code basée sur la programmation orientée objet.

1. Présentation du projet

Dans le cadre de ma formation en BTS CIEL, j'ai réalisé un projet consistant à développer une API REST en Python permettant de gérer une liste d'étudiants enregistrés dans une base de données MySQL nommée « ciel2027 ».

L'API a été développée avec le framework Flask. Elle reçoit des requêtes HTTP envoyées par un client comme Postman, traite les demandes puis communique avec la base de données grâce à la bibliothèque mysql.connector. Les principales opérations réalisées sont la consultation, l'ajout, la modification et la suppression d'étudiants.

La connexion à la base de données utilise l'adresse 127.0.0.1, l'utilisateur root et la base ciel2027.

2. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les actions que l'utilisateur peut effectuer avec l'API REST de gestion des étudiants.

L'utilisateur peut consulter la liste complète des étudiants, consulter un étudiant précis à partir de son identifiant, ajouter un nouvel étudiant, modifier les informations d'un étudiant ou supprimer un étudiant. Chaque action correspond à une fonctionnalité accessible par une route HTTP de l'API.

Ce diagramme permet donc de visualiser simplement les fonctionnalités principales du système et le rôle de l'utilisateur.

3. Architecture générale du projet

L'architecture du projet repose sur plusieurs éléments qui communiquent entre eux. L'utilisateur utilise Postman comme client HTTP pour envoyer des requêtes à l'API Flask.

Flask reçoit la requête et exécute le traitement correspondant à la route demandée. Lorsque les informations doivent être lues ou modifiées, l'application communique avec la base de données MySQL à l'aide de mysql.connector.

La base de données ciel2027 contient notamment la table « etudiants » avec les champs idetudiant, nom, prenom, email et telephone. Une fois le traitement terminé, Flask renvoie une réponse au format JSON.

4. Diagramme de séquence – Consultation d'un étudiant

Le diagramme de séquence représente les échanges entre les différents composants lors de la consultation d'un étudiant.

Dans l'exemple présenté, l'utilisateur demande à Postman l'adresse GET /v2/etudiants/1. Postman transmet alors la requête à l'API Flask. Flask exécute une requête SQL de type SELECT auprès de MySQL afin de rechercher l'étudiant correspondant à l'identifiant demandé.

MySQL renvoie le résultat à Flask. L'API transforme ensuite les informations en réponse JSON et les transmet à Postman, qui les affiche à l'utilisateur. Cette séquence montre clairement le cheminement d'une requête depuis le client jusqu'à la base de données puis le retour de la réponse.

5. Description des routes de l'API

L'API utilise le protocole HTTP et renvoie les résultats au format JSON. Plusieurs méthodes HTTP sont utilisées selon l'opération à effectuer.

La route GET /v2/etudiants/ permet de récupérer la liste complète des étudiants. La route GET /v2/etudiants/{id} permet de rechercher un étudiant à partir de son identifiant.

La méthode POST `/v2/etudiants/` permet d'ajouter un nouvel étudiant. Les informations sont transmises dans le corps de la requête, puis enregistrées dans la base de données.

La méthode PUT `/v2/etudiants/{id}` permet de modifier les informations d'un étudiant existant. Enfin, la méthode DELETE `/v2/etudiants/{id}` permet de supprimer un étudiant.

L'API utilise notamment les codes HTTP 200 OK lorsque l'opération est réussie, 201 Created lorsqu'un étudiant est créé et 404 Not Found lorsqu'un étudiant demandé n'existe pas.

6. Diagramme CRUD

Le diagramme CRUD présente les quatre opérations principales réalisées par l'API sur la base de données.

CREATE correspond à l'ajout d'un étudiant avec la méthode POST. READ correspond à la consultation des données avec la méthode GET. UPDATE correspond à la modification d'un étudiant avec la méthode PUT. DELETE correspond à la suppression d'un étudiant avec la méthode DELETE.

Ces quatre opérations permettent de gérer les données de la table `etudiants` de manière complète.

7. Diagramme de classes – Version 3

Dans la version 3 du projet, une classe Database est prévue afin de centraliser les opérations réalisées sur la base de données. Cette organisation permet de séparer davantage la gestion de la connexion et des requêtes SQL du code principal de l'API.

La classe Database contient les paramètres de connexion tels que `host`, `user`, `password` et `database`. Elle prévoit également des méthodes permettant de se connecter, récupérer les étudiants, récupérer un étudiant par son identifiant, créer, modifier et supprimer un étudiant.

Une méthode de connexion utilisateur est également prévue dans le cadre de l'ajout d'un système d'authentification.

8. Historique des versions

La version 1 correspond à la première mise en place de l'API et des opérations CRUD avec des routes utilisant le préfixe `/v1/`.

La version 2 ajoute une meilleure gestion des erreurs avec `try/except`, notamment pour les erreurs 404 et 500. Des contrôles supplémentaires sont également réalisés sur les données reçues et le `rowcount` permet de vérifier si une modification ou une suppression a réellement été effectuée. Les routes utilisent alors le préfixe `/v2/`.

La version 3 est prévue pour améliorer l'organisation du projet. Elle doit notamment intégrer une table `user`, une authentification avec POST `/login`, une classe Database, une programmation davantage orientée objet et une séparation du code dans un fichier `db.py`.

9. Tests réalisés avec Postman

Plusieurs tests ont été réalisés avec Postman afin de vérifier le bon fonctionnement des différentes routes de l'API.

Les tests comprennent la récupération de tous les étudiants avec GET `/v2/etudiants/`, la récupération d'un étudiant existant avec GET `/v2/etudiants/1`, ainsi que la recherche d'un étudiant inexistant avec GET `/v2/etudiants/999`, qui doit produire une réponse 404 Not Found.

Des tests POST, PUT et DELETE ont également été effectués afin de vérifier respectivement l'ajout, la modification et la suppression d'un étudiant. Ces essais permettent de contrôler le comportement de l'API et les codes HTTP retournés.

10. Compétences mises en œuvre

9.1 Résultats des tests avec Postman

Capture des tests réalisés sur les différentes routes de l'API REST.

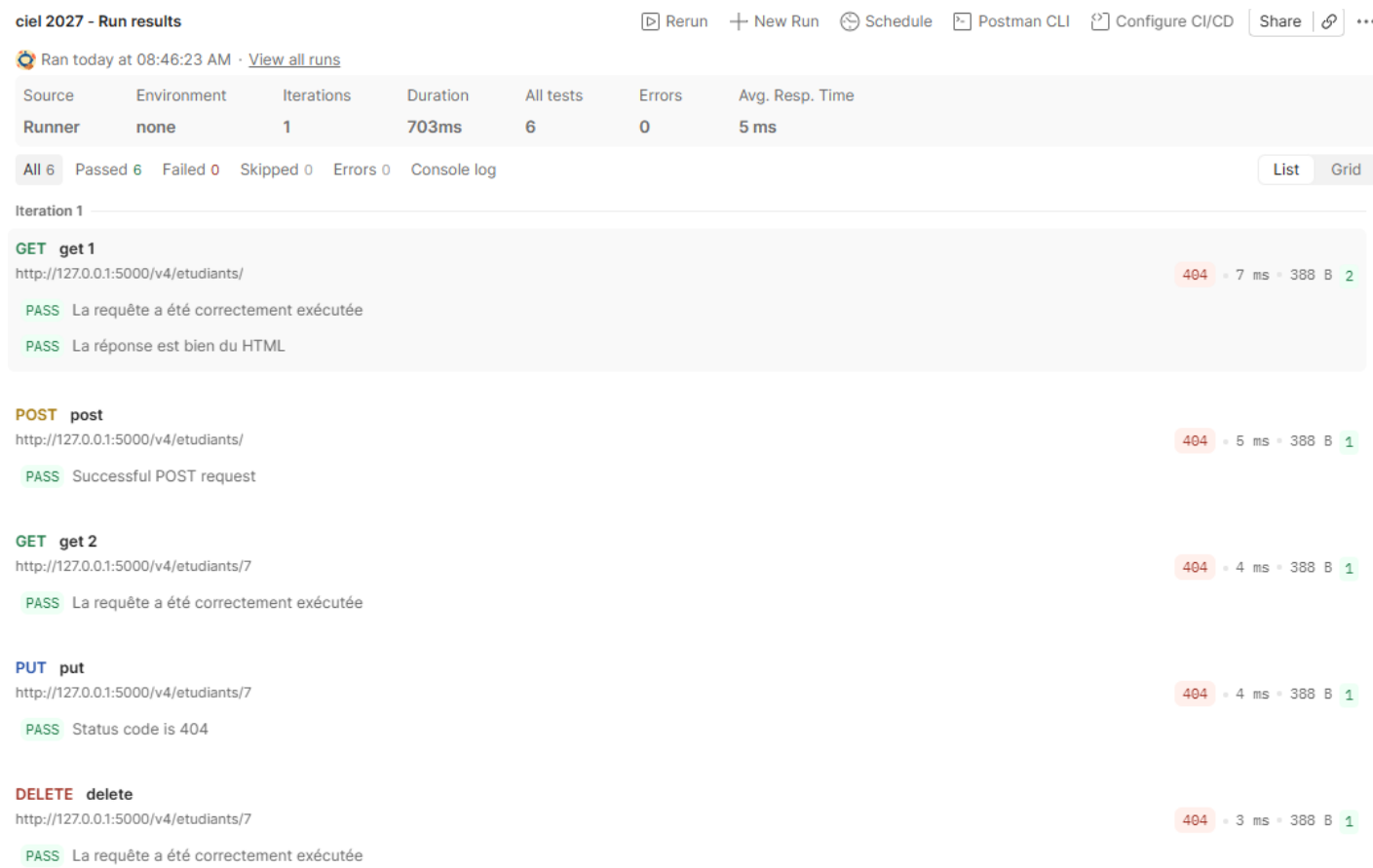


Figure : résultats des requêtes GET, POST, PUT et DELETE exécutées avec Postman.

Ce projet m'a permis de mettre en pratique plusieurs compétences liées au développement informatique et aux réseaux.

- Développement en Python.
- Utilisation du framework Flask.
- Connexion et gestion d'une base de données MySQL.
- Utilisation des requêtes SQL SELECT, INSERT, UPDATE et DELETE.
- Création et utilisation d'une API REST.
- Utilisation du format d'échange JSON.
- Utilisation de Postman pour tester une API.
- Gestion des erreurs et des codes HTTP.
- Notions de programmation orientée objet pour l'évolution du projet.

11. Conclusion

Ce projet m'a permis de développer une API REST complète en Python avec Flask et MySQL. J'ai pu mettre en œuvre les principales opérations CRUD et apprendre à faire communiquer une application Python avec une base de données.

Les différents tests réalisés avec Postman m'ont permis de vérifier le fonctionnement des routes et de mettre en place une gestion des erreurs. Le projet peut ensuite être amélioré avec la version 3, notamment grâce à l'authentification des utilisateurs, à une meilleure organisation du code et à l'utilisation d'une classe Database.

Cette réalisation m'a permis de mieux comprendre le fonctionnement d'une API REST, les échanges HTTP, les bases de données relationnelles et l'organisation d'un projet Python.